

MemCatalog: A DBMS-style Memory Management System for LLM Agents

Anonymous Author(s)

Abstract

LLM agents accumulate memory that grows to hundreds of thousands of tokens per user, and in some settings to millions of tokens. The dominant treatment of this memory remains an unmanaged string collection: raw sessions concatenated into an LLM prompt, or dense vectors retrieved by static similarity. This is the pre-DBMS era of agent memory, in which systems can *store* but cannot *manage* their memory. We introduce MemCatalog, a DBMS-style memory management system for LLM agents that lifts agent memory into first-class managed data. MemCatalog comprises a foundational framework and two novel optimizations built upon it. (i) A full query-optimization stack, consisting of a catalog, a zero-supervision planner, and three physical operators (index scan, hash-aggregate, and sort-merge), dispatches each query to the matching physical plan. (ii) A *Second-Order Memory* logs per-user access patterns and reshapes catalog priorities, a workload-to-storage feedback loop transferred from classical DBMS self-tuning to agent memory for the first time. (iii) A *Cost-Aware Executor* allocates the LLM call budget by plan class and repurposes self-consistency as a cost signal, not a quality voter. Across eight long-term memory benchmarks, MemCatalog matches or improves the strongest prior memory system on every one, by an average of 2.6 points and by up to 5.6 points on LME-s. On MemBench it further reduces LLM calls by 50.7% relative to a Uniform- $K=3$ reference while attaining higher accuracy.

CCS Concepts

• **Information systems** → **Data management systems**; **Query optimization**; • **Computing methodologies** → *Natural language processing*; Multi-agent systems.

Keywords

Agent memory, query optimization, large language models, self-tuning systems, cost-based optimization

ACM Reference Format:

Anonymous Author(s). 2026. MemCatalog: A DBMS-style Memory Management System for LLM Agents. In *Proceedings of Proceedings of the 2026 International Conference on Management of Data (SIGMOD '26)*. ACM, New York, NY, USA, 14 pages. <https://doi.org/XXXXXXX.XXXXXXX>

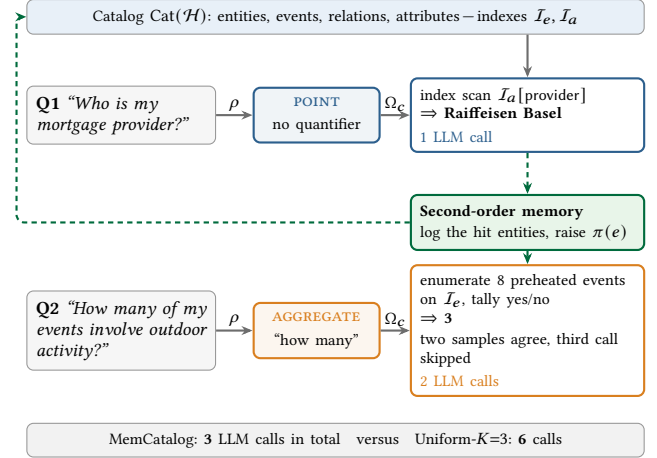


Figure 1: Motivating example. MemCatalog selects a physical plan for each query (**POINT** or **AGGREGATE**), serves it with the minimum LLM-call budget, and logs hit entities into second-order memory so that the next query reads them first. The interaction costs three LLM calls in total, where a Uniform- $K=3$ baseline would spend six.

1 Introduction

LLM agents now hold conversations that span weeks, and the interaction history of a single user grows to hundreds of thousands of tokens, in some deployments to millions [7, 26, 29]. Answering a question against that history is the core function of a personal agent, and it is also the function that current systems serve least well. The obstacle is not the cost of storing the history but the absence of any declared structure over it, since the history arrives as free-form natural language. A system therefore cannot tell a single-fact lookup apart from a count over scattered evidence, and cannot treat them differently. Every read becomes the same operation, a similarity scan over an opaque token stream followed by one generation call. The system never knows what a query is asking, what the same user has asked before, or what an answer ought to cost. Agent memory today is stored but not managed. Figure 1 contrasts this behavior with MemCatalog on a two-query interaction, and

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD '26, Bengaluru, India

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM. ACM ISBN 978-1-4503-XXXX-X/2026/06 <https://doi.org/XXXXXXX.XXXXXXX>

Figure 2 previews the accuracy MemCatalog reaches across eight long-term memory benchmarks.

State-of-the-Art Approaches and Their Limits. Three lines of work attack this problem. Each improves how memory is written and leaves untouched how it is read, which is where a query is actually served. *Long-context prompting* and *RAG variants* [16, 18] push raw history into the prompt or retrieve top- k chunks by dense similarity. A point lookup and a cross-entity count then follow an identical retrieval-and-generation pipeline. *Hierarchical memory systems* such as MemGPT [26] page facts between context tiers, which decides *where* a fact lives but not *how* to compute over facts of different logical shapes. *Structured memory systems* such as Mem0 [7] and Zep [29] invest heavily in write-time entity and graph extraction, yet both fall back to one dense-retrieval strategy at read time, and neither records which stored memories actually produced correct answers for a given user. The three lines differ in representation but share a common shape. Prior systems supply a storage layer and sometimes an access path, but no planner that distinguishes plan classes, no operator library that computes each class differently, and no cost model that prices an answer before paying for it.

The database community closed analogous gaps four decades ago by lifting flat files into *managed data*, where schemas expose record shape, optimizers select physical plans over a declared algebra, and cost models allocate execution resources [12, 31]. Recent systems carry that abstraction into LLM analytics over typed tables [27], but the line stops where the data stops being typed. Agent memory is unstructured, its schema is latent in natural language, and its queries are user questions instead of SQL. No existing system transfers the optimization stack to memory itself.

Challenges. Carrying the stack across that gap raises three obstacles. In each case a direct remedy exists and fails for a concrete reason.

C1. Recovering a Plan Space without a Schema or Labels. Query optimization presupposes that the system can name the shape of a query, but agent memory offers neither a relational schema to plan against nor labeled examples to learn a planner from. Asking the LLM to tag each query would work, yet it spends an extra call on every question and turns the plan itself into a sampled variable, which is an undesirable property for a component every downstream decision depends on. Training a supervised planner is unavailable because no benchmark ships query-type labels.

C2. Learning a User without Gold Supervision. A memory system should get better at a specific user as that user keeps asking, but nothing in the deployment reveals which stored records deserve priority. Loading every historical entity into the prompt exceeds the context window after a few sessions. Ranking by recency is available without any supervision, yet it ranks by the wrong quantity, because an entity a user returns to across months is old precisely because the relationship is long-standing. The signal that does matter, which records participate in correct answers, lies on the read path, and prior systems instrument only the write path.

C3. Paying for Reasoning in Proportion to Difficulty. Replicated sampling buys accuracy on hard questions and wastes it on easy ones, yet difficulty is not observable before the answer exists. Lowering the budget uniformly sacrifices the queries that needed it, and an

LLM-based difficulty predictor reintroduces the per-query call the budget was meant to save.

Our Proposal. We present MemCatalog, a memory management system that answers all three obstacles with one move, by giving agent memory the storage, planning, and execution separation of a DBMS. Three mechanisms follow from that separation.

A Typed Catalog with Zero-Supervision Plan Selection. MemCatalog lifts sessions into a catalog of entities, events, relations, and attributes, materialized lazily on first access, and routes each question to one of three physical plans using only surface form, the quantifier and comparative markers in the question and the shape of the answer choices. The planner costs no LLM call and needs no labels, which resolves C1. Three operators then execute the chosen plan over the catalog indexes. A count enumerates and tallies where a comparison extracts and sorts, instead of both being handed to one generic prompt.

Second-Order Memory over the Read Path. MemCatalog records, per user, which plan classes have been issued and which catalog entities appeared in accepted answers, then folds that record back into index priority and grounding-block composition. The system observes the read path that prior work discards, which resolves C2, and it transfers workload-driven index recommendation [5] from self-tuning databases to agent memory.

Agreement as a Cost Signal. MemCatalog assigns a call budget per plan class and then truncates it adaptively by comparing two independent samples. Reading sample agreement as a stopping signal is known [1, 17], but it is applied to one query at a time and therefore cannot avoid the first two samples. MemCatalog determines from the plan class alone that a point lookup will not benefit from a second sample, and it therefore allocates across classes before adapting within one, which resolves C3.

EXAMPLE 1. *Alice asks her agent “How many of my events involve outdoor activity?” after six earlier questions in the same session. A Full-Context baseline scans the whole history and answers 5, miscounting evidence scattered across sessions, where the correct answer is 3. A graph memory retrieves six outdoor-related nodes but never resolves the outdoor predicate, and the model guesses 4. MemCatalog routes the question to its aggregation plan, surfaces the event entities that the six earlier questions already marked as relevant, enumerates each candidate with an explicit yes-or-no decision, and answers 3. The two samples it draws agree, and it stops after two calls. Each of the three failure modes above is removed by a different one of the three mechanisms. Table 5 gives the full trace.*

Contributions. (1) We formulate agent memory management as query optimization and build MemCatalog, which supplies the catalog, planner, and operator library that prior memory systems lack (Sections 3 and 4). (2) We introduce second-order memory, which closes a feedback loop from the read path back to physical storage and lets a memory system improve on a user without supervision (Section 5). (3) We combine class-level budget allocation with agreement-based stopping, which together reach a cost point that neither mechanism attains alone (Section 6). (4) We evaluate MemCatalog on eight long-term memory benchmarks and eight LLM backbones, where it matches or improves the strongest prior memory system on every benchmark while using fewer LLM calls

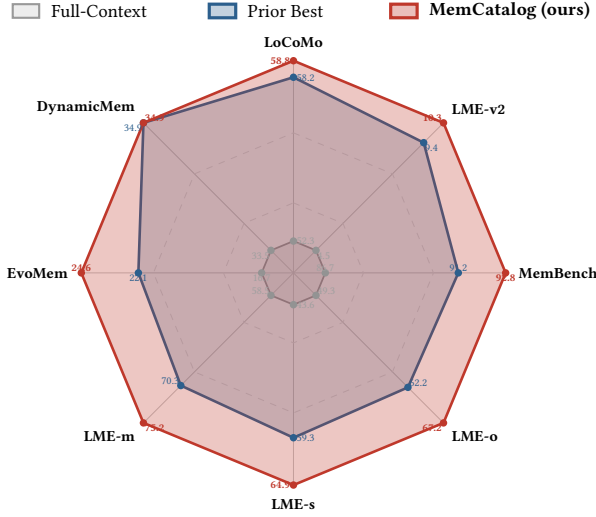


Figure 2: Per-benchmark accuracy on 8 long-term memory benchmarks. Each axis uses its own linear scale: Full-Context maps to the innermost ring, MemCatalog to the outermost regular octagon, and Prior Best falls at its position between them, so that within-benchmark differences among the three systems are directly visible. Raw accuracy (%) is annotated at every vertex. The prior-best system varies per benchmark (MemBench: Mem0; LME-v2: LongMemEval-Fw; the remaining six: Zep), matching Table 2.

than a uniform sampling reference of equal plan depth, and we release code, harness, and per-entry outputs for full reproduction¹.

2 Preliminaries

This section fixes notation, defines the agent memory management problem, describes how any memory-augmented LLM agent processes a query, and lists the three access primitives that structure the rest of the paper.

2.1 Interaction History and Memory Query

Interaction History. An LLM agent interacts with a user across a sequence of *sessions* $\mathcal{H} = \{s_1, \dots, s_N\}$, where $s_i = (t_i, \text{txt}_i)$ is a timestamp t_i paired with a natural-language transcript txt_i of mixed user and assistant turns. Sessions are read-only after they close, and $\mathcal{H}$ is unbounded and grows monotonically over time. In our benchmarks a single user’s history reaches up to 1.5M tokens spread over tens of sessions [21, 39], well beyond any current context window at inference time.

Memory Query. A memory query is a triple $q = (Q, C, \tau)$, where Q is a natural-language question, C is an optional set of answer choices (empty for open-ended questions), and τ is the timestamp at which the query is issued. The agent must return an answer $\hat{a}$ derivable from the history available at that moment, $\mathcal{H}_{\leq \tau} = \{s_i : t_i \leq \tau\}$. Answers are scored against a gold reference, with accuracy or F1 computed on $\hat{a}$ (Section 7.1).

¹Materials will be made publicly available upon acceptance.

2.2 Problem Statement

Given a stream of queries $q_1, q_2, \dots$ over an evolving $\mathcal{H}$, an *agent memory management system* produces answers $\hat{a}_1, \hat{a}_2, \dots$ under three simultaneous objectives.

- (O1) **Accuracy.** Maximize the fraction of $\hat{a}_t$ that match the gold answer for q_t .
- (O2) **Bounded per-query cost.** Keep the per-query LLM cost $\text{cost}(q_t)$ bounded and manageable as $|\mathcal{H}|$ grows, letting the system scale to long histories and heavy usage.
- (O3) **Improvement over time.** Later queries on the same $\mathcal{H}$ should improve, or at least not degrade, given the information gathered from earlier ones.

Prior memory systems have focused overwhelmingly on (O1) [7, 26, 29, 39]: their headline results report accuracy on a single test query per user, with cost and per-user adaptation treated implicitly or not at all. MemCatalog treats (O2) and (O3) as first-class objectives on par with (O1), and enforces each by a dedicated architectural layer, the executor for cost (Section 6) and the second-order memory for per-user improvement (Section 5).

2.3 How an LLM Agent Uses Memory

Regardless of the particular memory system, an LLM agent processes each user interaction along two coupled paths. A *write path* updates the memory store when a new session closes, and a *read path* answers a query using the accumulated store. Both paths spend LLM calls, and both are what MemCatalog instruments.

Every memory system, whatever its internal representation, runs the same two paths. A *write path* fires when a session closes, converting s_i into the store’s representation and appending it to a memory store $\mathcal{M}$. Systems differ here in representation alone, with long-context prompting keeping s_i verbatim, RAG embedding chunks of it, Mem0 extracting facts into a dictionary, and Zep building temporal-graph edges. A *read path* fires per query and proceeds in three steps that are common to all of them, (R1) retrieving a candidate set from $\mathcal{M}$, (R2) packing candidates into a grounding block that fits the context window, and (R3) reasoning over that block to produce $\hat{a}_t$.

Two observations about this structure motivate the rest of the paper. First, $\text{cost}(q_t)$ is dominated by R3, and replicated sampling multiplies it on every query without regard to whether the query needed the samples. Second, prior systems run an identical R1–R3 pipeline for every query and retain nothing from the outcome. Neither the shape of Q_t nor the history of this user’s earlier queries changes how the next one is served.

2.4 Memory Access Primitives

Empirically, natural-language queries against agent memory fall into three access patterns that differ in the shape of the computation they require. This taxonomy is the entry point for both the planner and the physical operator library of Section 4.

- **Point lookup (POINT).** Retrieve a single stored fact by attribute key, e.g., “Who is my mortgage provider?”. The answer is a single value from the catalog’s attribute index and does not require aggregation or comparison.

- **Aggregation (AGGREGATE)**. Compute a quantity (typically a count) over multiple records, e.g., “How many of my events involve outdoor activity?”. The answer requires enumerating candidates from the entity index and applying a predicate to each.
- **Comparison (COMPARE)**. Rank or order two or more records on a shared attribute, e.g., “Which lasted longer, the Basel trip or the Zurich conference?”. The answer requires extracting the compared key from each candidate and reducing it to an arg max.

These three classes have direct DBMS analogues (*index scan*, *hash-aggregate*, *sort-merge*) that Sections 4.2 and 6.1 exploit, and the LLM’s failure modes differ across them (reliable point lookup, undercount on scattered aggregation [14], and misalignment on cross-passage comparison). Prior memory systems collapse the three into one retrieve-then-generate pipeline and inherit all three failure modes. MemCatalog splits them and gives each its own operator.

3 An Overview of MemCatalog

This section traces one query end to end, so that the division of labor among the three mechanisms is visible before any of them is specified in detail. Figure 3 shows the resulting architecture, in which a framework supplies the plan and two optimization layers refine what that plan reads and what it costs.

Life Cycle of a Query. Given a query q , MemCatalog evaluates Eq. (1) in four steps. It first *plans* the query, mapping q to one of POINT, AGGREGATE, or COMPARE from surface form alone (Section 4.2). It then *grounds* the selected plan, resolving the catalog indexes $\mathcal{I}_e$ and $\mathcal{I}_a$ that the chosen operator will read, with the second-order memory reweighting those indexes by the per-user priority $\pi(e)$ so that entities this user has already confirmed appear first (Section 5.2). It then *executes*, issuing LLM calls under the operator’s fixed template while the executor decides how many calls the plan class warrants and stops early once two samples agree (Sections 6.1 and 6.2). It finally *records* the outcome, writing the entities that appeared in the accepted answer into the workload log $\mathcal{W}_u$ so that the next query starts from a better-ordered catalog (Section 5.1).

Interlocking of the Three Mechanisms. The ordering above is not a pipeline of independent stages, since each mechanism consumes something another produces. The executor cannot price a query until the planner has named its class. The planner cannot read a useful candidate set until the second-order memory has ranked the catalog. The second-order memory cannot update until the executor has committed to an answer worth recording. Removing any one therefore degrades the other two, which Section 7 confirms by ablation. The remainder of the paper takes the three in turn.

4 Memory as Managed Data

The framework lifts unstructured agent memory into managed data. Given a history $\mathcal{H} = \{s_i\}_{i=1}^N$ of natural-language sessions and a query stream $\{q_t\}_{t \geq 1}$ with $q_t = (Q_t, C_t, \tau_t)$, prior systems compute one retrieve-then-generate pipeline for every query, regardless of what Q_t actually asks. MemCatalog instead factors the computation into a catalog that gives $\mathcal{H}$ a schema (Section 4.1) and a planner that dispatches q_t to one of three typed physical operators (Section 4.2).

Formally,

$$\hat{a}_t = \underbrace{\Omega_c}_{\text{Physical Operator}} \left(\underbrace{q_t, \text{Cat}(\mathcal{H})}_{\text{Catalog}}, \underbrace{c = \rho(q_t)}_{\text{Plan Selection}} \right), \quad (1)$$

where ρ selects a plan class $c \in \{\text{POINT}, \text{AGGREGATE}, \text{COMPARE}\}$ and Ω_c is the physical operator that realizes class c . Alice’s personal assistant from Example 1 is the running illustration.

4.1 Lifting Sessions into a Typed Catalog

Query optimization requires that the system know the shape of what it stores, which raw sessions do not reveal. This subsection builds that knowledge, describing what the catalog extracts, when it pays to extract it, and what interface the extraction exposes upward to the planner.

Typed Records. The catalog $\text{Cat}(\mathcal{H})$ decomposes each session s_i into four typed record sets, namely *entities* $\mathcal{E}$ (named actors, objects, locations), *events* $\mathcal{V}$ (timestamped occurrences), *relations* $\mathcal{R} \subseteq \mathcal{E} \times P \times \mathcal{E}$ (predicate-linked entity pairs), and *attributes* $\mathcal{A} \subseteq \mathcal{E} \times K \times \text{Val}$ (entity–key–value triples). A single session in which Alice mentions opening a mortgage contributes the entities {Alice, Raiffeisen Basel}, the event (opened mortgage account, 2025-03-14), the relation (Alice, has-provider, Raiffeisen Basel), and the attribute (mortgage, rate, 2.4%). What was previously opaque text now has a schema visible to the planner.

Lazy Materialization. Records are populated only on demand. When a query first touches session s_i , a single templated LLM call runs an extractor that returns $(\mathcal{E}_i, \mathcal{V}_i, \mathcal{R}_i, \mathcal{A}_i)$, and the result is cached. Extraction therefore runs once per session instead of once per query, and its cost is amortized over every later query that touches the same session, in the way a DBMS pays for an index once and reuses it across a workload.

Access Paths. The catalog exposes two indexes to the planner. An entity index $\mathcal{I}_e$ maps each entity to its host sessions, and an attribute index $\mathcal{I}_a$ maps each attribute key to the triples (e, v, s_i) that carry it. These are the only interface through which physical operators read memory, which keeps the storage layer swappable and the retrieval ranking tunable by the second-order memory (Section 5).

4.2 Routing Queries to Typed Physical Operators

With the catalog in place, the remaining question is which computation a given query deserves. This subsection answers it in two steps, a selection rule that names the plan class without supervision and an operator library that realizes each class differently, and then bounds what happens when the selection rule is wrong. As Eq. (1) states, the planner first selects a plan class $c = \rho(q)$ and then executes the matching operator Ω_c .

Zero-Supervision Plan Selection. The planner $\rho : (Q, C) \rightarrow \{\text{POINT}, \text{AGGREGATE}, \text{COMPARE}\}$ uses only surface features $\phi_{\text{surf}} = \phi_{\text{surf}}(Q)$ (quantifier phrases, comparative or superlative markers) and choice-shape features $\phi_{\text{shape}} = \phi_{\text{shape}}(C)$ (numeric, entity, or



Figure 3: Overview of MemCatalog. A foundational framework (catalog, planner, physical operators) is augmented by two optimization layers: the Second-Order Memory, which closes a workload-to-storage feedback loop, and the Cost-Aware Executor, which combines plan-class budgeting with a stability short-circuit.

free-text), written as

$$\rho(q) = \begin{cases} \text{AGGREGATE}, & \phi_{\text{surf}} \in \text{QuantPhr} \vee \phi_{\text{shape}} = \text{num}, \\ \text{COMPARE}, & \text{else if } \phi_{\text{surf}} \in \text{CmpPhr}, \\ \text{POINT}, & \text{otherwise.} \end{cases} \quad (2)$$

The three predicates are checked in order, need no supervised labels or LLM call, and dispatch Alice’s questions correctly. Here *QuantPhr* is a fixed list of quantifier phrases (how many, the number of, count), *CmpPhr* a fixed list of comparative and superlative markers (more, longer, most), and *num* the case where the choices *C* are numeric. “Who is my mortgage provider?” contains no quantifier or comparative and falls through to *POINT*. “How many of my events involve outdoor activity?” matches how many and routes to *AGGREGATE*. “Which lasted longer, the Basel trip or the Zurich conference?” pairs longer with or and routes to *COMPARE*. This mirrors a DBMS query analyzer, which decides between an index probe, an aggregation, and a sort before generating any plan.

Robustness of Plan Selection. A misselected plan class is a performance concern rather than a correctness one, for two reasons. The three operators read the same catalog through the same two indexes, so every plan class retrieves admissible evidence and returns a well-formed answer regardless of which class was selected. And *POINT* is the fall-through branch of Eq. (2), so a query that matches no quantifier or comparative marker is served on the cheapest admissible path instead of paying for an operator it does not need. Section 7.8 measures the accuracy that plan selection itself contributes.

Physical Operator Library. Each plan class binds an access path to a fixed reasoning template, stated compactly as

$$\begin{aligned} \Omega_{\text{POINT}}(q) &= \text{LLM}(\text{template}_{\text{POINT}}(\mathcal{I}_a[k_q])), \\ \Omega_{\text{AGGREGATE}}(q) &= \{ \{ e \in \mathcal{I}_e[T_q] : \text{LLM}(\text{pred}_q, e) = \text{yes} \} \}, \\ \Omega_{\text{COMPARE}}(q) &= \arg \max_{e \in \{e_1, e_2\}} \mathcal{I}_a[k_q](e). \end{aligned}$$

POINT probes $\mathcal{I}_a$ for the requested key k_q and returns the value directly, an index scan. *AGGREGATE* enumerates candidate entities of the referenced type T_q from $\mathcal{I}_e$, marks each against the predicate with a bounded yes/no judgment, and emits an integer count. On the outdoor question it lists Alice’s eight events, tags them, and

emits Answer: 3. This is a hash-aggregate that makes the count explicit instead of leaving it implicit inside free-form generation. *COMPARE* extracts the compared key k_q (here, duration) for the two candidates from $\mathcal{I}_a$ and materializes both side by side, turning a long-context alignment problem into a two-value sort-merge.

Remarks. The three components compose into the standard DBMS triple. The catalog supplies statistics and access paths, the planner selects a plan, and the operator library executes it. Prior memory systems provide a storage layer and sometimes an index, but every query, whether a single-fact lookup or a cross-entity count, still follows one generic retrieve-then-generate pipeline. Making the plan class $c = \rho(q)$ in Eq. (2) explicit is the precondition for the two optimization layers that follow. The second-order memory tunes the indexes $\mathcal{I}_e, \mathcal{I}_a$ that a plan reads (Section 5), and the executor prices each plan class differently (Section 6). Neither layer is expressible without an explicit plan class.

5 Second-Order Memory over the Read Path

The framework of Section 4 evaluates each query in isolation. The hundredth query for a user is therefore served in the same way as the first. Prior memory systems share this limitation. They observe only the *write path*, which sessions arrived, and never the *read path*, which stored records answered which queries. MemCatalog closes this loop with a per-user workload log $\mathcal{W}_u$ (Section 5.1) and a feedback rule that reshapes the catalog’s physical organization (Section 5.2).

5.1 Recording the Read Path

Adapting storage to a user presupposes a record of what that user’s queries actually consumed. Prior systems never keep that record. This subsection specifies what MemCatalog logs, and it keeps the log small enough that logging does not itself become a scaling problem. For each user u , MemCatalog maintains a compact log of query outcomes

$$\mathcal{W}_u = \{ (q_t, \rho(q_t), E_t^+, \tau_t) \}_{t \leq T},$$

where $E_t^+ \subseteq \mathcal{E}$ is the set of catalog entities that appeared in the accepted answer $\hat{a}_t$ or its supporting evidence. The log carries three signals. The sequence of plan classes $\{\rho(q_t)\}$ describes *what*

the user asks. The sets $\{E_t^+\}$ record *what memory answers it*. Sessions left unread beyond a staleness horizon indicate *what memory is dead weight*. After Alice's first six queries, her log records a POINT-heavy class mix over financial and scheduling topics, hits on Raiffeisen Basel and her calendar events, and no reads on her older travel sessions.

Per-Entity Statistics. The log is statistical, not verbatim. For each catalog entity e we keep three running numbers only, an issue count $n(e) = |\{t \leq T : e \in q_t\}|$ (how often e has been mentioned by any query), an answer-hit count $h(e) = |\{t \leq T : e \in E_t^+\}|$ (how often e appeared in an accepted answer), and a last-hit time $\tau^*(e) = \max\{\tau_t : e \in E_t^+\}$. Storage is therefore $O(|\mathcal{E}|)$ per user and independent of query volume T . A DBMS keeps histograms and access counters instead of a replay log of every statement, for the same reason, since the optimizer needs the workload's distribution and not its transcript.

What Counts as an Accepted Answer. No gold labels exist at deployment time, and acceptance must therefore be defined operationally. MemCatalog marks an answer accepted when the executor commits to it under Eq. (5), which happens when two independent samples agree, and records only the entities the operator read in producing it. The score moves a ranking and never a decision, and a promoted entity changes which records the operator sees first, never what it computes or may return. The loop is a retrieval prior, not a cache of asserted facts.

Per-User Isolation. $\mathcal{W}_u$ and every quantity derived from it are strictly per-user, with no cross-user aggregation. A new user starts from a neutral prior and adapts as queries arrive.

5.2 Reshaping Storage from Observed Workload

A log is useful only if something reads it. This subsection turns the log into a single per-entity score and then spends that score on three decisions about where memory physically sits, leaving query semantics untouched throughout.

Workload-Driven Priority Score. Each catalog entity e carries a priority score

$$\pi(e) = \underbrace{\alpha \cdot h_r(e)}_{\text{used in answers}} + \underbrace{\beta \cdot r(e)}_{\text{used recently}} + \underbrace{\gamma \cdot c(e)}_{\text{matches the class}}, \quad (3)$$

where $h_r(e) = h(e)/n(e)$ is the hit rate on queries that mention e , $r(e) = \exp(-\lambda(\tau_{\text{now}} - \tau^*(e)))$ is a recency term with decay rate λ , and $c(e)$ is the fraction of queries mentioning e whose plan class matches e 's record type. The three weights are fixed at $(\alpha, \beta, \gamma) = (0.5, 0.3, 0.2)$ for every benchmark and backbone we report, with direct workload evidence given the dominant share and the remaining two terms acting as tie-breakers. We do not tune them per dataset, and Section 7.6 reports the effect of removing each term outright.

Storage-Level Actions. $\pi(e)$ is refreshed after every query and drives the physical organization of the catalog through three actions, governed by fixed system-wide thresholds. *Index warming* keeps the highest-priority entities by π in a resident cache that biases candidate retrieval. *Grounding-block composition* assembles the evidence block prepended to an operator's prompt from the top candidate entities by $\pi(e)$, and not from a static set. *Session*

eviction compresses or drops any session s_i whose entities all fall below a cold threshold π_{cold} . By Alice's seventh query, her calendar events sit at the top of the grounding block while the never-read travel sessions have been compressed away. When AGGREGATE then enumerates candidates for the outdoor question the eight relevant events are the first records it reads. The operator itself is unchanged, and the workload has reorganized only the catalog it reads.

All three actions operate on *where* memory sits, not on *what* a query means. The access paths $\mathcal{I}_e, \mathcal{I}_a$ simply return workload-reweighted results, and every physical operator benefits without issuing an additional LLM call.

Remarks. The priority score of Eq. (3) implements the self-tuning-index principle [5, 6] and the workload-adaptive stance of learned optimizers [22, 23, 42], transferred to agent memory, where the system observes the query log, updates statistics online, reshapes physical storage, and leaves query semantics untouched. In MemCatalog the recommender is $\pi(e)$, the log is $\mathcal{W}_u$, and the storage is the catalog. To our knowledge, no prior agent memory system closes this workload-to-storage loop. This is why prior accuracy on a given user stays flat as that user issues more queries (Section 7.7).

6 Cost-Aware Execution

A memory system that answers well but spends unboundedly does not scale to long histories and heavy usage. This section prices what MemCatalog spends and then shows how to spend less of it without giving up accuracy.

Accounting. MemCatalog issues LLM calls on both the write path (catalog extraction) and the read path (query reasoning). Write-path cost is paid once per session and amortizes across queries. The rest of this section concerns read-path cost, which dominates. The standard remedy for accuracy, self-consistency at $K=3$ [36], triples that cost on *every* query, regardless of whether the extra samples change the answer. Formally, the total cost over a workload of Q queries with class fractions $(p_{\text{POINT}}, p_{\text{AGGREGATE}}, p_{\text{COMPARE}})$ under fixed budgets $(K_{\text{POINT}}, K_{\text{AGGREGATE}}, K_{\text{COMPARE}})$ is

$$C = Q \cdot (p_{\text{POINT}} K_{\text{POINT}} + p_{\text{AGGREGATE}} K_{\text{AGGREGATE}} + p_{\text{COMPARE}} K_{\text{COMPARE}}). \quad (4)$$

A Uniform- $K=3$ reference sets $K_{\text{POINT}} = K_{\text{AGGREGATE}} = K_{\text{COMPARE}} = 3$, so $C = 3Q$.

The executor makes this budget an explicit, plan-aware decision, exploiting two independent sources of waste. One is the class of queries saturated at a single call (Section 6.1). The other is the individual query whose answer is stable before its budget is spent (Section 6.2). The two act on disjoint slices, and their savings compose.

6.1 Budgeting Calls by Plan Class

The first source of waste is structural, since one class of query never benefits from the samples it is given. This subsection assigns a budget per class and derives what that assignment saves as a function of workload composition. The executor pre-declares a per-class budget instead of a per-query one, setting $K_{\text{POINT}} = 1$ and $K_{\text{AGGREGATE}} = K_{\text{COMPARE}} = 3$. The asymmetry follows from what each plan computes, not from a heuristic. A POINT query resolves a single stored fact. Once (Alice, has-provider, Raiffeisen Basel) is

in the grounding block, replicated sampling can only re-read it, and a single call is both necessary and sufficient. AGGREGATE and COMPARE queries compose evidence across multiple records, as when the outdoor count must decide eight independent yes/no judgments, and there sampling genuinely reduces variance, so these classes keep the full budget. Substituting these budgets into Eq. (4), the saving against Uniform- $K=3$ is $\Delta C = 2p_{\text{POINT}}Q$. On MemBench with $p_{\text{POINT}} = 0.61$, this alone reduces C from 900 to 532, a 40.9% cut. The saving grows with the share of point queries and costs nothing in accuracy on exactly the queries it economizes, in line with the DBMS discipline of pricing an operator by what it must compute and not by a uniform per-tuple rate.

6.2 Short-Circuiting on Probe Agreement

The second source of waste lies inside the hard classes, where some instances are easy but indistinguishable from the rest before execution. This subsection recovers that distinction at no extra cost by reading the agreement between samples the system was going to draw anyway. Within the AGGREGATE/COMPARE budget, the executor decides adaptively whether the third call is worth making. It draws two independent samples p_1, p_2 under the operator's template, measures their token-level agreement $\text{agree}(\cdot, \cdot) \in [0, 1]$ against a threshold θ , and draws a third sample p_3 only when the two disagree, giving the rule

$$\hat{a} = \begin{cases} p_1, & \text{agree}(p_1, p_2) \geq \theta, \\ \text{Majority}(p_1, p_2, p_3), & \text{otherwise.} \end{cases} \quad (5)$$

Under Eq. (5) on Alice's outdoor question, both samples enumerate the same eight events and emit Answer: 3, so the agreement is high, the third call is skipped, and the query costs two. Had one sample returned 3 and the other 4, the low agreement would have triggered escalation, spending the third call where it can change the outcome. We fix $\theta = 0.6$ for all benchmarks and all backbones, and it is never retuned per dataset. Combined with plan-class budgeting, agreement-based stopping drives the total cost on MemBench from 532 to 444, a further 16.5% cut and 50.7% overall against the Uniform- $K=3$ reference.

Agreement as a Cost Signal. The reinterpretation here is the contribution. Self-consistency [36] samples K reasoning paths and votes, treating disagreement as *noise to average away* under a sampling budget fixed in advance. MemCatalog reads the same disagreement as *information about the query*. When two samples agree, the query is easy for the current model and grounding, and a third sample is unlikely to change the outcome. Agreement thus becomes a stopping rule, not a voting mechanism. Adaptive-Consistency [1] and early-stopping self-consistency [17] both halt sampling once drawn answers converge, but they operate on a single query in isolation and must draw at least two samples before learning anything. MemCatalog knows from Eq. (2) that a point lookup cannot benefit from a second sample at all, and it skips the extra sample entirely for the majority of the workload. Agreement-based stopping and class-based budgeting are therefore complementary, the first acting within a class and the second across classes, and Section 7.5 measures them separately.

Table 1: The eight benchmarks, grouped by the memory ability they stress. History length spans two orders of magnitude, which lets us separate gains that come from structure from gains that come from context length.

Benchmark	Memory ability	History
LoCoMo [21]	Multi-session dialogue	~9K
LME-v2 [39]	Entity-centric recall	~115K
MemBench [34]	Typed access primitives	~100K
LME-o [39]	Long-horizon retention	~40K
LME-s [39]	Long-horizon retention	~115K
LME-m [39]	Long-horizon retention	~1.5M
EvoMem [38]	Evolving user state	~50K
DynamicMem [40]	Evolving user state	~127K

7 Experiments

We organize the evaluation around eight questions. Exp-1 asks whether MemCatalog outperforms state-of-the-art memory systems across memory abilities. Exp-2 asks whether the advantage survives a change of backbone. Exp-3 asks which plan classes the advantage comes from. Exp-4 asks what the accuracy costs in LLM calls. Exp-5 asks how much each architectural layer contributes. Exp-6 asks whether the second-order memory improves a user over time. Exp-7 asks how accurate the zero-cost planner is and how sensitive the end-to-end result is to its two free parameters. Exp-8 asks how read-path cost behaves as history length grows.

7.1 Experimental Setup

Datasets. We use the eight long-term memory benchmarks in Table 1, chosen to separate three memory abilities that a single benchmark would confound. LoCoMo [21] and LME-v2 [39] test recall over multi-session dialogue. LME-o, LME-s, and LME-m [39] share one question style and scale the surrounding history by nearly two orders of magnitude, which lets history length be varied as a controlled factor. EvoMem [38] and DynamicMem [40] issue many questions per user over an evolving state, the only setting in which per-user adaptation becomes observable. MemBench [34] labels every one of its 300 queries with the access primitive it requires, and it is therefore the one benchmark on which a per-class decomposition is possible.

Baselines. We compare against seven systems in two groups. (1)

Prompting Baselines. **Full-Context** [18] places the entire history in the context window, and **RAG** [16] retrieves top- k passages by dense similarity. (2) *Memory Systems.* **MemGPT** [26] pages facts between context tiers. **Mem0** [7] consolidates facts into a dictionary through an ADD/UPDATE/DELETE loop, and **Mem0^g** adds entity relations. **Zep** [29] maintains a temporally-aware knowledge graph. **LongMemEval-Fw** [39] applies a three-stage indexing, retrieval, and reading pipeline. All seven are run from their released implementations under the same backbone, which leaves the memory design and not the model as the source of any difference.

Evaluation Metrics. Multiple-choice benchmarks are scored by exact match. The remaining benchmarks are scored by F1 against

the gold answer, normalized at the phrase level for LME-v2 and at the token level elsewhere. Phrase-level normalization credits a prediction only when it reproduces the gold phrase, so absolute F1 on LME-v2 sits far below the token-level scores, and the two scales are not comparable; within LME-v2 the ranking across systems is unaffected. Higher is better throughout, and every reported number is an accuracy on the benchmark’s own scale.

Backbone LLMs and Settings. Unless stated otherwise all systems run on GPT-5.5. Exp-2 additionally evaluates three further closed-source backbones [3, 11, 24] and four open-source backbones, selected to span a wide capability range at two deployment tiers. MemCatalog is implemented in Python against an OpenAI-compatible client. Every hyperparameter is fixed once and reused across all benchmarks and backbones, with no per-dataset tuning.

7.2 Exp-1. Accuracy Across Three Memory Abilities

We compare MemCatalog against all seven baselines on the eight benchmarks of Table 1, grouped by the memory ability each one stresses. Table 2 gives the comparison. MemCatalog matches or improves the strongest prior system on every benchmark, and the margin varies systematically with the memory ability the benchmark stresses.

Long-Horizon Memory. The margin is widest on the three Long-MemEval variants, where it reaches 4.9 to 5.6 points, and it does not shrink as history grows by nearly two orders of magnitude. The reason lies in what each system hands the model. A longer history enlarges the candidate set that similarity retrieval returns, so prior systems pass the model more passages and leave it to locate, order, and align the relevant facts inside the context window, the regime in which long-context models are known to undercount and misorder [14, 18]. MemCatalog reads the fields a query needs from the catalog indexes, so the grounding block stays small and already structured, and the model performs the reasoning step rather than the search. The size of that block is set by the plan class and not by $|H|$, which is why the margin persists at the largest scale.

Conversational Memory. The margin is narrowest on this group, at 0.6 points on LoCoMo and 1.6 on MemBench. These benchmarks ask mostly for facts stated in one place, the access pattern a single retrieval pass already serves well, so there is less headroom for a plan to recover. The result on LME-v2 is 10.3 against 9.4 for the strongest baseline, a 9.6% relative improvement on a strict phrase-level F1 scale on which every system scores in the single digits. What distinguishes MemCatalog on this group is that the accuracy and the cost advantage hold together: on MemBench it reaches the highest accuracy while issuing fewer LLM calls than the uniform reference, as Exp-4 shows.

Personalized State. The two state-inference benchmarks separate the systems least in absolute terms, with all seven baselines falling inside 5.4 points on EvoMem and 1.4 points on DynamicMem. These benchmarks issue many questions per user, and a system that treats each question independently answers the tenth exactly as it answered the first, which is what prior designs do: they record what arrives on the write path and never what succeeded on the read path. An aggregate accuracy therefore averages over the very

variable the second-order memory acts on. Exp-6 conditions on query position and recovers the effect that the aggregate hides.

Finding 1: MemCatalog matches or improves the strongest prior system on all eight benchmarks, and the margin is largest where a single retrieval pass is least sufficient, on long histories and on repeated questions from one user.

7.3 Exp-2. Larger Gains on Weaker Backbones

A system that supplies external structure should benefit every backbone, and the benefit should be largest where the backbone’s own capacity is most limited. To test this, we hold MemBench and the full MemCatalog configuration fixed and vary only the backbone, over four closed-source and four open-source models spanning a wide capability range. Table 3 collects the comparison.

MemCatalog improves every backbone, which rules out an explanation resting on one model’s inductive bias. The size of the improvement is what varies. It is smallest on the strongest backbone and largest on the weakest, and this ordering holds within each group separately. This pattern is consistent with the structure MemCatalog supplies substituting for capability the backbone lacks. A frontier model already holds more history in context and counts over it more reliably, so the incremental gain is smaller, but the gain remains positive even on the strongest backbone. The practical consequence is that MemCatalog widens the set of backbones on which long-term memory is usable, which matters most where frontier inference per query is not affordable.

Finding 2: The advantage of MemCatalog is not backbone-specific, and it grows as backbone capability falls, with external memory structure substituting for internal model capacity.

7.4 Exp-3. Gains Concentrate on Aggregation and Comparison

Exp-1 established that MemCatalog leads overall. To locate the source of that lead, we decompose MemBench accuracy by the access primitive each query requires, which MemBench labels directly. Table 4 breaks the accuracy out by class.

The decomposition reveals where the advantage originates. MemCatalog leads on all three plan classes, and the gains concentrate on aggregation and comparison, where an explicit plan replaces free-form reasoning. The separation appears on these two classes because they require the model to compose evidence across multiple records.

Aggregation. Counting fails for reasons that a better retriever cannot fix. When candidate items are scattered through a long context, models miscount even after retrieving every relevant item [14]. Prior systems retrieve passages and ask for a count, which leaves the count implicit inside free-form generation. MemCatalog makes it explicit, enumerating candidates, deciding each one against the predicate, then tallying. Each step is bounded, and the total rests on enumerable evidence instead of an estimate over context.

Table 2: Accuracy across eight long-term memory benchmarks. MemCatalog matches or improves the strongest prior system on every benchmark, and the margin is widest on the long-horizon and state-inference groups where a single retrieval pass is least sufficient.

Method	Conversational memory			Long-horizon memory			Personalized state	
	LoCoMo	LME-v2	MemBench	LME-o	LME-s	LME-m	EvoMem	DynamicMem
<i>Prompting baselines</i>								
Full-Context [18]	52.3	4.5	86.7	49.3	43.6	58.5	16.7	33.5
RAG [16]	53.1	4.9	87.8	50.8	45.1	59.1	17.3	34.1
<i>Memory systems</i>								
MemGPT [26]	54.5	6.3	89.7	54.1	48.4	61.1	18.0	34.3
Mem0 [7]	56.4	7.2	91.2	58.3	54.2	65.2	20.4	34.5
Mem0 ^g [7]	57.1	7.5	90.8	60.1	56.7	67.1	21.2	34.7
Zep [29]	<u>58.2</u>	8.1	90.4	<u>62.2</u>	<u>59.3</u>	<u>70.3</u>	<u>22.1</u>	<u>34.9</u>
LongMemEval-Fw [39]	57.3	<u>9.4</u>	90.1	61.7	58.8	69.4	21.6	34.6
<i>Ours</i>								
MemCatalog	58.8	10.3	92.8	67.2	64.9	75.2	24.6	34.9

Table 3: Accuracy on MemBench across backbones. Best prior is the strongest of the seven baselines of Table 2 re-run on that same backbone. The gain is smallest where the backbone is strongest, so the structure MemCatalog supplies substitutes for capacity the model does not have.

Backbone	Best prior	MemCatalog
<i>Closed-source</i>		
GPT-5.5	91.2	92.8
Claude-3.5-Sonnet	87.4	91.4
Gemini-1.5-Pro	85.6	89.7
GPT-4o	82.8	88.7
<i>Open-source</i>		
DeepSeek-V4	82.7	88.5
Llama-3.1-70B	79.8	86.5
Qwen3-14B	77.4	85.1
Qwen3-8B	73.4	81.3

Comparison. Comparison fails through misalignment rather than miscounting. The model must locate the compared attribute in two separate passages and align them before it can order them, and that alignment is what breaks on long histories. MemCatalog performs the alignment outside the model, reading the attribute from the typed index for each candidate and placing both values adjacent in the prompt. The remaining task is an ordering over two values.

Both failure modes live in how the model *processes* retrieved evidence, not in what the retriever returns, which is why improving retrieval alone does not close the gap and why the computation has to follow from the plan class.

To make the two failure modes concrete, Table 5 traces one aggregation query through MemCatalog and two baselines. Both baselines had the relevant material available, and in neither case did retrieval fail. The prompting baseline miscounts evidence spread across sessions, and the graph memory retrieves topically related

Table 4: Accuracy by plan class on MemBench. Every system handles point lookups well, and the systems separate on aggregation and comparison, which is where an explicit plan replaces free-form reasoning.

Method	POINT	AGGREGATE	COMPARE
<i>Prompting baselines</i>			
Full-Context [18]	90.9	64.4	84.2
RAG [16]	91.8	67.1	85.8
<i>Memory systems</i>			
MemGPT [26]	92.9	68.5	87.0
Mem0 [7]	93.9	70.2	88.1
Mem0 ^g [7]	94.1	<u>71.9</u>	85.4
Zep [29]	<u>94.3</u>	71.2	85.4
LongMemEval-Fw [39]	93.8	70.8	84.7
<i>Ours</i>			
MemCatalog	94.6	78.0	91.2

Table 5: One aggregation query traced through three systems. The two baselines fail differently, and each failure is removed by a different layer of MemCatalog.

System	Retrieval and reasoning	Calls	Answer
Full-Context	count left implicit	1	5
Zep [29]	predicate unresolved	1	4
<i>MemCatalog, layer by layer</i>			
Planner routes on the quantifier marker			
Second-order memory fronts this user’s resolved entities			
Operator decides each candidate, then tallies			
Executor withholds the third call on agreement			
MemCatalog	explicit per-item decision	2	3

nodes without resolving the predicate the question asks about. MemCatalog removes the two failures with different layers. The planner

Table 6: Calls per query on MemBench. The two MemCatalog rows share an accuracy column, which isolates the short-circuit as a pure cost mechanism and attributes the accuracy to the plan and not to the budget.

Configuration	Calls per query			Total	Acc.
	POINT	AGGREGATE	COMPARE		
Uniform, one call	1.00	1.00	1.00	300	86.7
Uniform, three calls	3.00	3.00	3.00	900	89.1
MemCatalog, budget only	1.00	3.00	3.00	532	92.8
MemCatalog	1.00	2.30	2.18	444	92.8

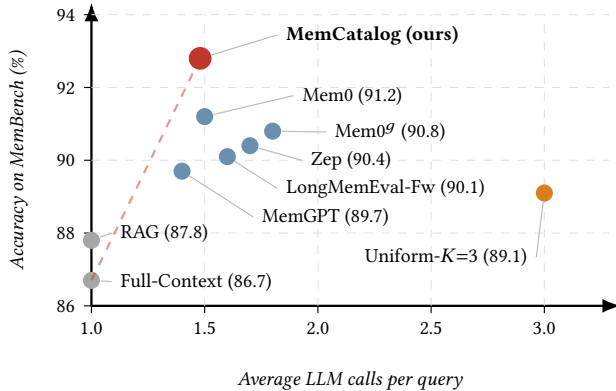


Figure 4: Cost-accuracy Pareto on MemBench (GPT-5.5, $n=300$). Each point is a system's average LLM calls per query (x-axis) versus its accuracy (y-axis). MemCatalog occupies the upper-left corner: strictly higher accuracy than every prior memory system and strictly lower cost than the Uniform- $K=3$ reference. No prior system Pareto-dominates MemCatalog.

selects the aggregation plan through Eq. (2), the second-order memory fronts the entities this user's earlier questions resolved, the operator records an explicit decision per candidate before tallying, and the executor withholds its third call once two samples agree.

Finding 3: The accuracy advantage concentrates on aggregation and comparison, where an explicit plan replaces free-form reasoning over scattered evidence.

7.5 Exp-4. Lower Cost at Higher Accuracy

Accuracy bought with extra LLM calls is not an architectural result. To separate the two, we measure the LLM calls each system issues per query on MemBench and place cost and accuracy on the same axes. Cost is counted on the read path for every system, so a system that spends a retrieval or rewriting call before generating is charged for it. Write-path extraction is a one-time per-session cost that every structured system amortizes over the queries touching that session, including the lazy materialization of MemCatalog, and

Table 7: Removing one component at a time. Each layer contributes on a distinct axis, accuracy for the framework, accuracy under repeated queries for the second-order memory, and cost for the executor, confirming the three are complementary.

Configuration	MemBench		LME-v2	
	Acc.	Calls	Acc.	Calls
MemCatalog	92.8	444	10.3	602
<i>Framework</i>				
w/o planner	89.1	900	8.8	903
w/o physical operators	87.9	900	8.3	903
w/o catalog	86.7	900	7.6	903
<i>Second-order memory</i>				
w/o workload log	92.1	444	8.6	602
w/o index warming	92.5	444	9.3	602
w/o session eviction	92.8	444	10.1	651
<i>Cost-aware executor</i>				
w/o plan-class budget	92.5	756	10.3	903
w/o stability short-circuit	92.8	532	10.3	723

it is excluded uniformly rather than charged to one group. We additionally include two reference configurations that fix the sampling budget uniformly at one call and at three calls per query, which isolates the effect of *allocating* a budget from the effect of its *size*.

Figure 4 gives the result. MemCatalog occupies the upper-left region, where no evaluated system attains both lower cost and comparable accuracy. The two groups miss it for different reasons. Prior memory systems are inexpensive and lose accuracy on the queries a single pass cannot answer. The uniform three-call reference recovers part of that accuracy and remains dominated, because it spends its extra samples on the point lookups that never required them.

Table 6 separates the two mechanisms behind this position. Plan-class budgeting acts across queries, assigning the single-call plan to the point lookups that form the majority of the workload. Stability short-circuiting, the rule of Eq. (5), acts within the remaining queries and stops early on those whose two samples already agree. The first mechanism draws from the easy class and the second from the easy instances of the hard classes, and the slices are disjoint, which lets the savings compose. The accuracy column settles what the savings cost: it is unchanged between the two MemCatalog configurations, so the short-circuit removes 88 of 532 calls without giving up a single answer, and the accuracy of MemCatalog is attributable to the plan and the operator rather than to the sampling budget.

Finding 4: MemCatalog is Pareto-optimal against every evaluated system, and the short-circuit lowers its cost at unchanged accuracy, which attributes the gain to plan selection instead of extra calls.

Table 8: Accuracy against the position of a query in a user’s own sequence, evaluated on users with ≥ 10 queries so that all four buckets are populated. Only MemCatalog improves as a user keeps asking. On DynamicMem it starts below the static rankers in the first bucket, where its workload log is still empty, and overtakes them once the log carries signal. Absolute levels differ from Table 2, which averages over all users.

Method	Query position for one user			
	1–3	4–6	7–9	10+
<i>EvoMem</i>				
Full-Context	16.7	17.0	16.8	16.5
Mem0 [7]	20.1	20.5	20.4	20.2
Zep [29]	<u>21.8</u>	<u>22.2</u>	<u>22.1</u>	<u>22.0</u>
MemCatalog	22.2	24.4	26.1	27.8
<i>DynamicMem</i>				
Full-Context	28.7	28.8	28.6	28.4
Mem0 [7]	<u>29.4</u>	29.8	29.6	29.3
Zep [29]	29.9	<u>30.0</u>	<u>29.9</u>	<u>29.8</u>
MemCatalog	29.1	30.3	31.7	32.8

7.6 Exp-5. Three Layers, Three Complementary Contributions

The three layers of MemCatalog were motivated by three different obstacles. This experiment confirms that each layer contributes independently by removing one component at a time and measuring both accuracy and cost on MemBench and LME-v2, a multiple-choice benchmark and a free-form one. Table 7 collects the ablations.

Each layer contributes on a distinct axis, confirming they are complementary. Removing any framework component costs accuracy and simultaneously removes the plan that the budget depends on, so cost rises to the uniform setting. Within the framework the ordering is informative. Removing the catalog is worse than removing the planner, since a planner that routes correctly still has nothing typed to read. The second-order memory contributes most on LME-v2, where a user’s repeated questions benefit from read-path feedback, and its three components are not equally active on both benchmarks: session eviction leaves MemBench untouched, whose histories stay within the cache, while on LME-v2 it saves 49 calls. In the executor the two mechanisms differ in kind. Plan-class budgeting lowers cost on both benchmarks and also contributes 0.3 points on MemBench, since a point lookup that is sampled three times can be outvoted on the one answer the index already supplies. Stability short-circuiting changes no accuracy figure on either benchmark and is a cost mechanism alone.

Finding 5: Each layer contributes on a distinct axis, accuracy for the framework, accuracy under repeated queries for the second-order memory, and cost for the executor, confirming they are complementary.

7.7 Exp-6. Accuracy That Grows With a User’s History

The second-order memory predicts a specific and falsifiable behavior. Accuracy on one user should rise as that user asks more questions, and no system without a read-path feedback loop should show the same rise. To test this we restrict to users with at least ten queries, so that all four position buckets are populated, condition on the position of a query within its own user’s sequence, and report accuracy per position bucket, holding backbone, prompt, and benchmark fixed so that position is the only variable. Table 8 bins the accuracy by position. Because the evaluation population is a subset of the full test set, the absolute accuracy levels in each column differ from Table 2; the within-table ranking and the monotone trend are the subject of this experiment.

The predicted separation appears on both benchmarks. MemCatalog improves monotonically across position buckets, gaining 5.6 points on EvoMem and 3.7 on DynamicMem from the first bucket to the last, while no baseline moves by more than 0.5 points in either direction and none shows a monotone trend, since none of them records which stored entries produced accepted answers and none has a signal that could vary with position. The starting point is equally informative. In the first bucket the workload log is still empty, so MemCatalog has no second-order signal yet and begins level with the static rankers, below Zep by 0.8 points on DynamicMem. It overtakes them from the second bucket onward and leads by 3.0 points by the last. A mechanism that learns from answered queries should hold no advantage before any query has been answered, and this is the crossover that behavior produces.

The improvement cannot be attributed to the model or the prompt, both of which are identical across buckets. It follows the priority score of Eq. (3), which accumulates evidence that an entity participates in answers this user accepts. By the later buckets the grounding block handed to the operator is composed largely of entities with a demonstrated history of being queried, where a static ranker still offers entities selected by similarity to the current question alone. This is the read-path signal of Section 5, and Exp-5 confirms that removing the workload log removes the effect.

Finding 6: Only MemCatalog improves as a user continues asking, which isolates read-path feedback as the mechanism and shows that prior systems are structurally unable to exhibit this behavior.

7.8 Exp-7. Classification Quality and Parameter Stability

Exp-5 identified the planner as the largest single contributor to accuracy. This experiment validates its design along two dimensions, classification quality and parameter stability. We measure classification accuracy against the class labels MemBench supplies, comparing the surface-rule planner against an LLM planner that spends one call per query and against a fixed planner that assigns every query to POINT. We then perturb the short-circuit threshold and the leading weight of the priority score over wide intervals and re-measure end-to-end accuracy. Table 9 covers both.

Table 9: Plan-selection quality and parameter stability on MemBench. The upper block compares the zero-supervision planner against an LLM planner that spends one call per query and against a fixed planner. The lower block varies the two parameters MemCatalog fixes system-wide around their defaults $\theta=0.6$ and $\alpha=0.5$, holding all else equal.

Planner	Class acc.	Planner calls	End-to-end acc.
Surface rules (ours)	91.4	0	92.8
LLM planner	95.1	1	93.2
Always POINT	61.3	0	87.5
<i>Parameter perturbation</i>			
$\theta \in [0.4, 0.8]$	end-to-end acc. 92.3–92.9		
$\alpha \in [0.3, 0.7]$	end-to-end acc. 92.2–92.8		

Three observations follow from the table. First, the surface-rule planner reaches 91.4% classification accuracy at zero LLM cost. Second, classification quality has already saturated with respect to what the system does with it: an LLM planner classifies better, at 95.1%, yet buys only +0.4 end-to-end accuracy for one extra call on every query, which is 300 additional calls against a total of 444, and a gap of 0.4 is no larger than the spread the parameter perturbations below produce. Paying two-thirds more calls for a difference of that size is not a trade this workload rewards. Third, the fixed planner locates where the value of plan selection actually sits: routing every query to POINT costs 5.3 points end-to-end, an order of magnitude more than separates any two planner variants, so what matters is that the plan class is chosen at all rather than how finely it is chosen. The lower block perturbs the two parameters MemCatalog fixes system-wide, the short-circuit threshold θ and the leading weight α of the priority score, across intervals spanning half of each parameter’s usable range. End-to-end accuracy stays within 0.6 points over both, which leaves the reported result independent of any parameter search.

Finding 7: The surface-rule planner reaches 91.4% classification accuracy at zero cost, an LLM planner cannot convert better classification into materially better answers, and end-to-end accuracy is stable across wide parameter intervals.

7.9 Exp-8. Cost Independent of History Length

A memory system is only interesting if its cost survives the growth that motivates it. The three LongMemEval variants share one question style and scale the surrounding history by nearly two orders of magnitude, from 40K to 1.5M tokens, which isolates history length as the varied factor. We measure calls per query, end-to-end latency, and accuracy at each scale against the prompting baseline, whose prompt necessarily carries the history it searches. Table 10 tracks all three.

Cost separates the two systems along the length axis. Calls per query are flat for both, at 1.9 for MemCatalog and 1.0 for Full-Context, so latency is where the difference appears. Latency grows 20.7× for Full-Context, from 2.3s to 47.6s, and 1.09× for MemCatalog, from 3.2s to 3.5s. At the shortest history MemCatalog is the

Table 10: Behavior as history grows on the three Long-MemEval variants. Read-path cost is governed by the plan class and not by history length, so it stays flat while the prompting baseline grows with the context it must carry.

History	Calls / query	Latency (s)	Accuracy
<i>Full-Context</i>			
~40K (LME-o)	1.0	2.3	49.3
~115K (LME-s)	1.0	5.8	43.6
~1.5M (LME-m)	1.0	47.6	58.5
<i>MemCatalog</i>			
~40K (LME-o)	1.9	3.2	67.2
~115K (LME-s)	1.9	3.4	64.9
~1.5M (LME-m)	1.9	3.5	75.2

slower of the two, by 0.9s, because catalog access is a fixed overhead that a single prompting call does not pay. That overhead does not grow, so it is already repaid at 115K, where MemCatalog answers in 3.4s against 5.8s, and at 1.5M it is 13.6× faster. What the table shows is not a smaller constant but a different asymptote. Accuracy moves with the difficulty of each variant for both systems, and the ranking does not: MemCatalog leads at all three scales, by 17.9, 21.3, and 16.7 points, with no erosion as history grows.

The asymmetry is structural and follows from where each system spends. A prompting baseline must carry the history it searches, so its prompt grows with $|H|$ and its latency grows with it. The read-path cost of MemCatalog is set by the plan class through Eq. (4), which depends on the shape of the query and not on the size of the store, and adding sessions changes what the catalog contains without changing what a query costs to answer.

Finding 8: Read-path cost is governed by the plan class rather than by history length, which turns the growth of the store from a linear cost into a constant one and leaves the accuracy advantage of MemCatalog intact at every scale.

8 Related Work

8.1 Agent Memory Systems and Benchmarks

The dominant thread in agent memory research treats memory as a *storage-side* abstraction. MemGPT [26] and its framework successor Letta [25] draw inspiration from operating-system memory hierarchies. They treat the LLM context window as a fast tier and external stores as slow tiers, with paging between them managed by the agent via function calls. Their contribution is the movement policy across tiers, not the query-time algebra over the stored data. Mem0 [7] designs a write-time pipeline that extracts, evaluates, and consolidates salient facts through an ADD/UPDATE/DELETE/NOOP loop, and its graph variant Mem0^g adds entity–relation edges. On the read side Mem0 falls back to dense-retrieval top- k , without a planner or any plan-space optimization. Zep [29], powered by the Graphiti engine, models memory as a temporally-aware knowledge graph over episodes, entities, and communities. It is tuned for retrieval accuracy and latency, but every query still goes through

the same retrieval strategy. Further work such as A-MEM [41], AriGraph [2], HippoRAG [13], MemoRAG [28], and hierarchical-retrieval methods such as RAPTOR [30] and GraphRAG [10] add autonomous memory evolution, biologically-inspired retrieval, and global-memory augmentation. Skill-library agents such as Voyager [35] treat learned skills as reusable memory. All of these systems enrich the storage layer, as chronicled by a recent survey [20]. None provides a query planner over typed physical operators, a cost model that budgets LLM calls by plan class, or a workload-to-storage feedback loop. MemCatalog fills these three gaps in Sections 4, 6, and 5.

Long-term memory evaluation has matured in parallel. LoCoMo [21] introduces multi-session dialogue at scale, and LongMemEval [39] covers five memory abilities up to 1.5M tokens while contributing the three-stage pipeline our LongMemEval-Fw baseline follows. Further benchmarks include MemoryBank [45], PerLTQA [9], DialSim [15], and LongBench [4].

8.2 DBMS Foundations for LLM Systems

Two complementary DBMS lines meet in this paper. First, a body of work applies DBMS abstractions to LLM analytics over *typed tabular data*. LOTUS [27] defines the first declarative model for AI-based tabular transformations such as semantic filter, join, top- k , and group-by, with gold algorithms and cost-optimized execution plans that achieve up to 1,000 $\times$ speedup. DocETL [32], UQE [8], and SUQL [19] address related settings for unstructured documents and hybrid structured/unstructured search. These works share the core intuition of MemCatalog, namely to lift LLM operations to a queryable algebra, but their target data is structured (DataFrames, tables). Agent memory is unstructured, its schema is latent in natural language, and its queries are user questions instead of SQL. To our knowledge MemCatalog is the first system to transfer the full query-optimization stack (planner, physical operators, cost model, and statistics) to agent memory.

Second, three founding lines of RDBMS research supply the pillars MemCatalog borrows, namely cost-driven access-path selection [31], extensible physical operator libraries [12], and workload-driven self-tuning [5, 6] that learned optimizers [22, 23, 42] later extended. The planner, the operator library, and the second-order memory of MemCatalog play these three roles in turn. The adaptation is not mechanical, since the operators reason over natural language instead of tuples and the cost unit becomes an LLM call, but the separation of concerns transfers intact.

8.3 LLM Reasoning Primitives

Chain-of-Thought [37], Self-Consistency [36], ReAct [44], Reflexion [33], and Tree-of-Thoughts [43] form the reasoning substrate on which the physical operators of MemCatalog build. A line of work reduces the cost of self-consistency by adapting how many samples a query receives, halting once drawn answers agree [1, 17]. These methods decide within a single query and must draw at least two samples before any decision is possible. MemCatalog composes with them from the opposite direction, using the plan class to decide how much budget a query is eligible for before sampling begins, which removes the extra sample entirely for the class that cannot benefit from it (Section 6).

9 Conclusion

We introduced MemCatalog, a DBMS-style memory management system for LLM agents. MemCatalog organizes agent memory as managed data under a full query-optimization stack of a catalog, a zero-supervision planner, and three physical operators, and augments this foundation with two layers, a *Second-Order Memory* that lets storage adapt to per-user workload and a *Cost-Aware Executor* that allocates its call budget by plan class and by sample agreement. On eight long-term memory benchmarks and eight LLM backbones, MemCatalog matches or improves the strongest prior memory system on every benchmark, by up to 5.6 points on LME-s. On MemBench it further reduces LLM calls by 50.7% relative to a Uniform- $K=3$ reference while attaining higher accuracy, and its second-order memory is the only mechanism among the evaluated systems whose accuracy rises monotonically across successive queries from the same user. The classical DBMS separation of storage, planning, and execution therefore transfers cleanly to agent memory, and each of the three layers contributes measurably. We view MemCatalog as a first step toward treating LLM agent memory as first-class managed data, a setting in which four decades of query-optimization results become applicable.

Future work will explore multi-user catalog sharing under privacy constraints, tighter integration of MemCatalog with retrieval-native architectures that pre-index by plan class, and extensions of the physical-operator library to multi-hop reasoning that currently falls back to AGGREGATE enumeration.

References

- [1] Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. 2023. Let's Sample Step by Step: Adaptive-Consistency for Efficient Reasoning and Coding with LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. arXiv:2305.11860.
- [2] Petr Anokhin, Nikita Semenov, Artyom Sorokin, Dmitry Evseev, Mikhail Burtsev, and Evgeny Burnaev. 2024. AriGraph: Learning Knowledge Graph World Models with Episodic Memory for LLM Agents. *arXiv preprint arXiv:2407.04363* (2024).
- [3] Anthropic. 2024. The Claude 3.5 Sonnet Model Card. <https://www.anthropic.com/news/claude-3-5-sonnet>.
- [4] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2023. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. *arXiv preprint arXiv:2308.14508* (2023).
- [5] Surajit Chaudhuri and Vivek Narasayya. 1997. An Efficient Cost-Driven Index Selection Tool for Microsoft SQL Server. In *VLDB*.
- [6] Surajit Chaudhuri and Gerhard Weikum. 2007. Self-Tuning Database Systems: A Decade of Progress. In *VLDB*.
- [7] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint arXiv:2504.19413* (2025).
- [8] Hanjun Dai, Bethany Yixin Wang, Xingchen Wan, Bo Dai, and Dale Schuurmans. 2024. UQE: A Query Engine for Unstructured Databases. *arXiv preprint arXiv:2407.09522* (2024).
- [9] Yiming Du, Hongru Wang, Zhengyi Zhao, Bin Liang, Baojun Wang, Wanjuan Zhong, Zezhong Wang, and Kam-Fai Wong. 2024. PerLTQA: A Personal Long-Term Memory Dataset for Memory Classification, Retrieval, and Synthesis in Question Answering. *arXiv preprint arXiv:2402.16288* (2024).
- [10] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130* (2024).
- [11] Google DeepMind. 2024. Gemini 1.5: Unlocking Multimodal Understanding across Millions of Tokens of Context. <https://arxiv.org/abs/2403.05530>.
- [12] Goetz Graefe. 1993. Query Evaluation Techniques for Large Databases. *Comput. Surveys* 25, 2 (1993), 73–169.
- [13] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. *arXiv preprint arXiv:2405.14831* (2024).

- [14] Cheng-Ping Hsieh, Simeng Sun, Samuel Krizan, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models? *arXiv preprint arXiv:2404.06654* (2024).
- [15] Jiho Kim, Woosong Chae, Yeonsu Kim, Seokchan Cho, Yoonseo Kim, Kyungjae Kim, Yohan Choi, and Jaegul Choo. 2024. DialSim: A Real-Time Simulator for Evaluating Long-Term Multi-Party Dialogue Understanding of Conversational Agents. *arXiv preprint arXiv:2406.13144* (2024).
- [16] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.
- [17] Yiwei Li, Peiwen Yuan, Shaoxiong Feng, Boyuan Pan, Xinglin Wang, Bin Sun, Heda Wang, and Kan Li. 2024. Escape Sky-high Cost: Early-stopping Self-Consistency for Multi-step Reasoning. In *International Conference on Learning Representations (ICLR)*.
- [18] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the Middle: How Language Models Use Long Contexts. *arXiv preprint arXiv:2307.03172* (2023).
- [19] Shicheng Liu, Jialiang Xu, Wesley Tjanganaka, Sina J. Semnani, Chen Jie Yu, and Monica S. Lam. 2023. SUQL: Conversational Search over Structured and Unstructured Data with Large Language Models. *arXiv preprint arXiv:2311.09818* (2023).
- [20] Junyu Luo et al. 2026. From Storage to Experience: A Survey on the Evolution of LLM Agent Memory Mechanisms. *arXiv preprint arXiv:2605.06716* (2026).
- [21] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:2402.17753.
- [22] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2020. Bao: Learning to Steer Query Optimizers. *arXiv preprint arXiv:2004.03814* (2020).
- [23] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. 2019. Neo: A Learned Query Optimizer. *arXiv preprint arXiv:1904.03711* (2019).
- [24] OpenAI. 2023. GPT-4 Technical Report. <https://arxiv.org/abs/2303.08774>.
- [25] Charles Packer, Sarah Wooders, et al. 2024. Letta: A Framework for Stateful LLM Agents with Memory Management. Successor of MemGPT; <https://github.com/letta-ai/letta>.
- [26] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560* (2023).
- [27] Liana Patel, Siddharth Jha, Melissa Pan, Carlos Guestrin, and Matei Zaharia. 2024. Semantic Operators: A Declarative Model for Rich, AI-based Data Processing. *arXiv preprint arXiv:2407.11418* (2024).
- [28] Hongjin Qian, Zheng Liu, Kelong Mao, Yujia Zhou, and Zhicheng Dou. 2024. MemoRAG: Boosting Long Context Processing with Global Memory-Enhanced Retrieval Augmentation. *arXiv preprint arXiv:2409.05591* (2024).
- [29] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv preprint arXiv:2501.13956* (2025).
- [30] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. In *ICLR*. arXiv:2401.18059.
- [31] P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access Path Selection in a Relational Database Management System. In *ACM SIGMOD*. 23–34.
- [32] Shreya Shankar, Tristan Chambers, and Aditya G. Parameswaran. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. In *ICLR*. arXiv:2410.12189.
- [33] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *NeurIPS*. arXiv:2303.11366.
- [34] Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. 2025. MemBench: Towards More Comprehensive Evaluation on the Memory of LLM-based Agents. In *Findings of the Association for Computational Linguistics: ACL 2025*. arXiv:2506.21605. doi:10.18653/v1/2025.findings-acl.989
- [35] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291* (2023).
- [36] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *ICLR*. arXiv:2203.11171.
- [37] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *NeurIPS*. arXiv:2201.11903.
- [38] Tianxin Wei, Naveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. 2025. Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory. *arXiv preprint arXiv:2511.20857* (2025).
- [39] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2024. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. *arXiv preprint arXiv:2410.10813* (2024).
- [40] Wenya Xie, Shengming Zhou, Zelin Li, Pouya Parsa, Shuang Zhou, Xinheng Ding, Chinmay Arvind, and Guanchu Wang. 2026. DynamicMem: A Long-Horizon Memory Benchmark in Real-World Settings. *arXiv preprint arXiv:2606.22877* (2026).
- [41] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. *arXiv preprint arXiv:2502.12110* (2025).
- [42] Zongheng Yang, Wei-Lin Chiang, Sifei Luan, Gautam Mittal, Michael Luo, and Ion Stoica. 2022. Balsa: Learning a Query Optimizer Without Expert Demonstrations. *arXiv preprint arXiv:2201.01441* (2022).
- [43] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *NeurIPS*. arXiv:2305.10601.
- [44] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *ICLR*. arXiv:2210.03629.
- [45] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. MemoryBank: Enhancing Large Language Models with Long-Term Memory. In *AAAI*. arXiv:2305.10250.